BlindBeat

Project proposal – No-code software development
Lara Normand – 15 October 2026 (final version)

BlindBeat is a Telegram bot for playing blind tests with friends, in a private chat or in a group. It plays 30-second clips from Spotify playlists, checks everyone's guesses and keeps the score, with three game modes.

Tool: n8n | **Channel:** Telegram (@lara_blindtest_bot) | **Database:** Google Sheets
APIs: Spotify, Deezer, OpenRouter | **Level reached:** 3

1. The need

Friends who want a quick music game in their group chat have no simple way to play one: someone has to run the music, someone keeps score, and everyone argues about who answered first.

What we do today instead: one person plays songs from Spotify on their phone, everyone shouts answers, and the score lives on a piece of paper or in someone's head. Blind test apps exist, but everyone has to download them, and they use their own song lists, not the music the group actually listens to.

What the bot changes: anyone in the chat links a Spotify playlist, the bot plays a clip, everyone types their guess, and the bot checks the answers and keeps the score. No app to install, and the songs come from the group's own playlists.

There are three game modes:

- **Classic (/play):** guess the title of a song from the group playlist.
- **Year (/year):** guess the release year. Exact year = 3 points, within 2 years = 2, within 5 years = 1.
- **Whose playlist? (/who):** each friend adds their own playlist with /mine, the bot plays a song from one of them, and you guess whose playlist it came from.

How a game works: /start shows a menu with one button per mode. After each round the next song plays automatically, /skip reveals the answer, /score shows the scores of the current game and /stop ends it with the final ranking.

2. The trigger

The main trigger is a Telegram Trigger. It runs every time someone sends a message to the bot: a command (/start, /menu, /play, /year, /who, /playlist, /mine, /skip, /score, /stop, /help) or a guess.

To test it, I just message the bot myself, in private and in a test group. I kept a list of test messages that I resend after each change:

Test message	**What it checks**
/play, then the right title	The normal case: clip sent, guess accepted, point saved, next song plays.
/play, then "wifey ridem" (typo)	The code accepts a close answer without any AI call.
/play, then a messy guess	The AI judge decides when the code is not sure.
/play, then a wrong title	The bot says "try again" and the round stays open.
/skip	The bot reveals the answer and plays the next song.
/year, then a year 2 off	Points are calculated from the distance (+2).
/who with only 1 player	The bot asks for at least 2 players.
"hello" with no round started	The bot replies "send /play to start" (only in private).

/playlist with no link	The bot sends the usage message.
/stop	Final scores of this game only, then the game ends.

3. Architecture

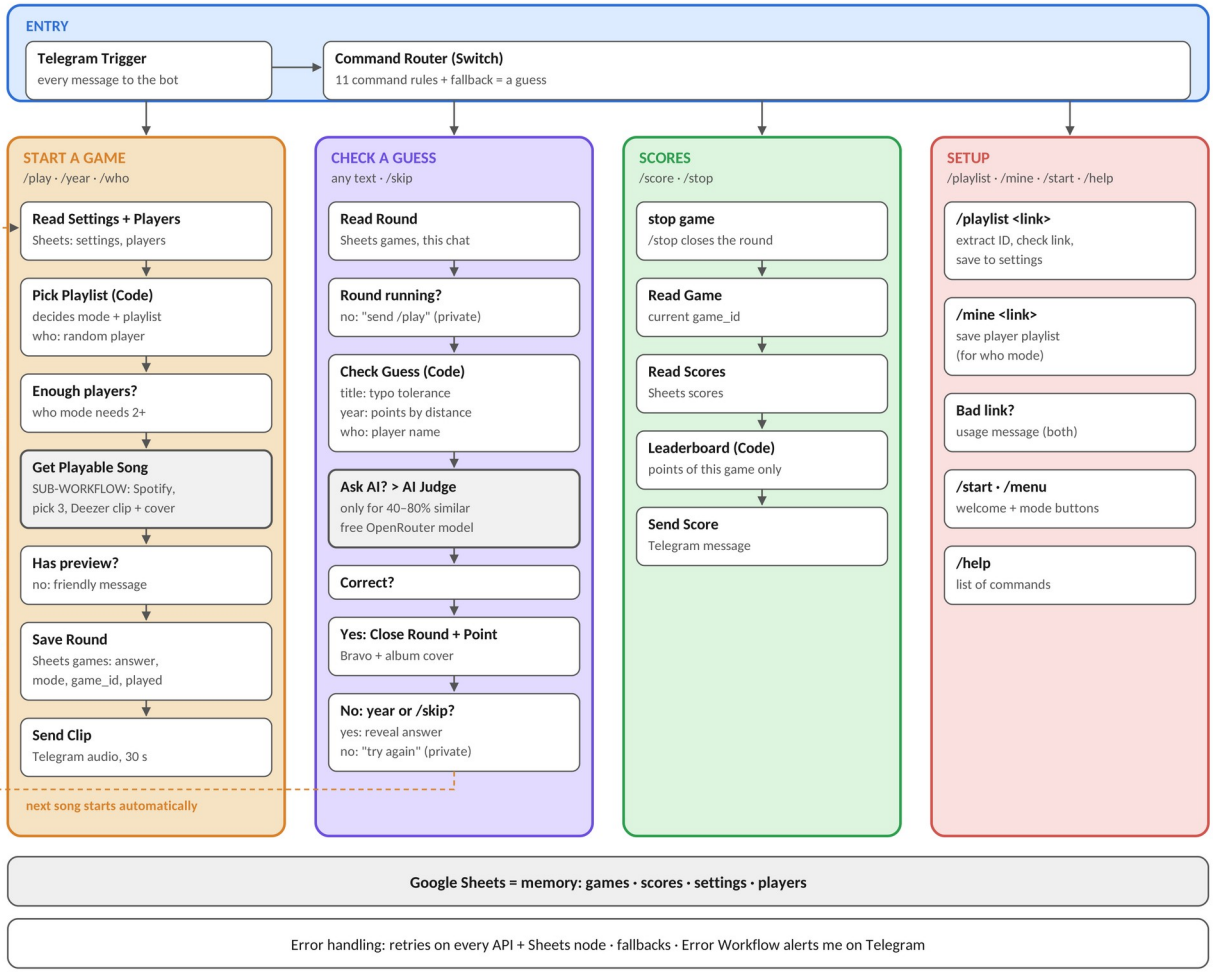


Figure 1 – Final architecture in 5 parts, same colours as the n8n canvas (3 workflows, 49 nodes in the main one)

Every message goes through one Switch node that sends it to the right branch. All three modes use the same sub-workflow to get a song, so one piece of logic is reused and tested on its own. The AI judge is only used when the code cannot decide, so most guesses need no AI call. Google Sheets is the memory between two messages: the round saved when the clip is sent is read back when the guess arrives.

The project has three workflows: **Blind Test Bot** (main), **Get Playable Song** (sub-workflow) and **BlindBeat Error Handler** (error workflow).

4. Level and what was delivered

I reached Level 3: a sub-workflow, error handling with retries and fallbacks, an error workflow, several services chained together and data saved between messages.

I built the classic mode first from start to finish, then added the rest on top. Everything in the original plan was delivered, including the "Whose playlist?" mode that was only a bonus. I also added things that came up while testing:

- **Continuous game:** the next song plays by itself after each answer.
- **Group mode:** wrong guesses stay silent in groups, so only the winner gets a reply.

- **/skip and "try again":** a wrong title no longer ends the round.
- **Scores per game:** each new game starts at 0, /stop shows the final ranking.
- **No repeated songs** within a game.
- **Album cover** shown with the answer, and a **start menu** with one button per mode.

5. Technical requirements

Requirement	Where I use it
If or Set node	If nodes: has a preview, round running, correct answer, end of round, private chat, enough players, valid link. Set nodes to extract the playlist ID from a link.
Code node	Picking the mode and playlist, picking 3 songs without repeats, cleaning titles, checking the guess (typo tolerance, year points, player name), reading the AI answer, building the leaderboard.
Router or If/Switch	One Switch with 11 command rules; anything else goes to the fallback output, which is treated as a guess.
API connection	Spotify for the playlists, the Deezer API (HTTP Request) for the 30-second clip and the album cover, OpenRouter for the AI judge.
Message trigger	Telegram Trigger on @lara_blindtest_bot.
Database	Google Sheets with four tabs: games (current round per chat), scores, settings (playlist per chat) and players (playlists for /who).
AI node	An LLM chain with a free OpenRouter model. It is only called when the guess is 40–80% similar to the title, and answers correct or not.
Sub-workflow	"Get Playable Song": takes a playlist, picks 3 candidate songs, finds the clip on Deezer and returns the first playable one with title, artist, year and cover.
Error handling	Retry on fail on Spotify, Deezer, OpenRouter and Sheets nodes. 3 candidate songs in case Deezer misses one. If the AI fails, the guess counts as wrong. If the cover fails, the answer is sent as text. An error workflow sends me a Telegram alert with the failing node.

6. Possible improvements

- Buttons under each clip (Skip, Score, Stop) instead of typed commands.
- Only the person who started a game can stop it or change the mode.
- A real database instead of Google Sheets if many groups played at the same time.